Algoritmos e Estruturas de Dados

3ª Série

(Problema)

GenomeSequencing

N. 51447 Ricardo Pinto

N. 51445 José Saldanha

N. 51484 Vasco Pereira

Licenciatura em Engenharia Informática e de Computadores

Semestre de Verão 2023/2024

05/06/2024

Índice

Conteúdo

1. INTRODUÇÃO **2**

2. ANÁLISE DE OCORRÊNCIA DE MENÇÕES EM TWEETS **3**

2.1 ANÁLISE DO PROBLEMA 3

2.2 ESTRUTURAS DE DADOS 4

2.3 ALGORITMOS E ANÁLISE DA COMPLEXIDADE 4

3. AVALIAÇÃO EXPERIMENTAL **6**

4. CONCLUSÕES **8**

REFERÊNCIAS **8**

1. Introdução

Neste relatório, vamos abordar o tema do problema da 3º série: sequência de um genoma, GenomeSequencing. Para este problema, pretendeu-se desenvolver uma aplicação que permite criar um grafo de De Bruijn e, com esse grafo criado, pretende-se criar um percurso Euleriano.

O objetivo deste problema é através de um dado k inteiro e um ficheiro com a listagem k-mers, obter o grafo de De Bruijn e o respetivo percurso euleriano.

O relatório está dividido em 4 partes:

- Aprofundação sobre o que é o problema e como foi abordado;
- Avaliação experimental do problema;
- Conclusões tiradas;
- Referências utilizadas para o relatório.

2. Análise de ocorrência de menções em tweets

Resumindo o conteúdo da secção 2, na secção 2.1 é descrito o problema e a abordagem encontrada para o resolver. Na secção 2.2 apresentam-se as estruturas de dados utilizadas. Por fim, a secção 2.3 contém alguns dos principais algoritmos utilizados, como os mesmos consultam/atualizam as estruturas de dados e a análise da sua complexidade.

2.1 Análise do problema

Para este problema, considerámos que cada ficheiro de *input* era constituído por várias linhas, em que cada uma é constituída por um conjunto de 3 bases nucleotídicas (As, Ts, Cs e Gs), por exemplo ATG, AGG, TTT, etc. Para retirar as informações deste ficheiro, precisamos que o programa leia as linhas que compõem todo o ficheiro. Para fazermos o grafo, teremos de implementar uma função que crie esse mesmo grafo. Para isto utilizamos a classe criada para o segundo exercício desta série, e com esta classe, fizemos com que ele adicionasse ao grafo os vértices das duas primeiras bases e das duas últimas bases. No fim, fizemos com que ele adicionasse uma aresta utilizando como parâmetro os vértices adicionados. Para o percurso Euleriano, utilizamos o grafo criado e um vértice inicial.

2.2 Estruturas de Dados

As estruturas de dados utilizadas para o problema foram a lista imutável, a lista mutável, o grafo criado no exercício 2 desta série e uma *stack*. A lista é uma estrutura de dados cujos dados são armazenados de forma ordeira, não podendo escrever na mesma. Podemos, apenas, ler os dados incluídos nela. A lista mutável tem um comportamento similar ao da lista imutável, contudo com a lista mutável podemos, para além de ler dados, escrever dados, podendo adicionar elementos à lista ou retirar elementos da lista. Na figura 1, podemos ver a sua realização.

Lista Imutável / Lista Mutável

	Braço	Pernas	Peito	Tronco	Mão	Cabeça	Dedos
Índices	0	1	2	3	4	5	6

Figura 1: Exemplo da estrutura de dados lista imutável e mutável. Caso a lista seja imutável, não podemos alterar o conteúdo, caso seja mutável, podemos alterar o conteúdo, adicionando ou removendo elementos da lista

O grafo criado por nós no exercício 2 desta série funciona como um grafo normal, em que podemos ver as adjacências a um vértice e saber se um vértice pertence ao grafo. Na figura 2, podemos ver um exemplo de um grafo.

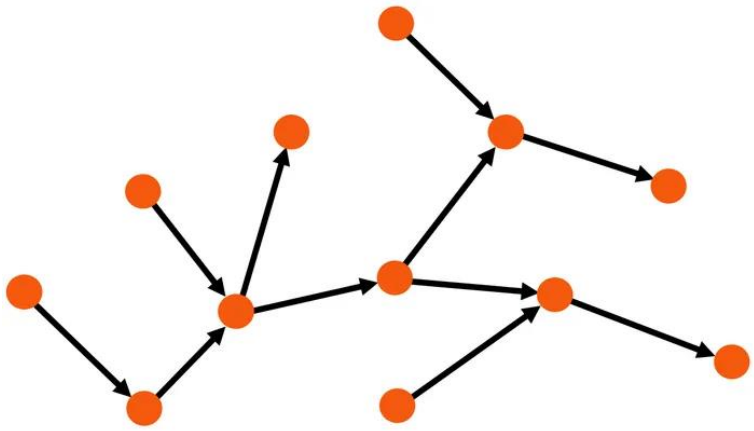

Figura 2: Exemplo da estrutura de dados grafo. Cada bola representa um vértice e cada seta representa uma aresta. Neste caso este grafo é um grafo orientado.

2.3 Algoritmos e análise da complexidade

O algoritmo é feito na função de criação do grafo. É um algoritmo simples, em que fazemos um ciclo for, onde por cada elemento de uma lista kMers, adicionamos ao grafo um vértice com as

duas primeiras bases e outro vértice com as duas últimas bases e, no fim, adicionamos uma aresta usando estes vértices como parâmetro. No final, retornamos o grafo já completo. Este algoritmo apresenta uma complexidade temporal *O(n*k)*, em que *n* representa o número de elementos dentro da lista kMers e *k* representa o valor escolhido para k. E uma complexidade espacial igual a *O(n+m)*, uma vez que o algoritmo armazena cada kMer como um vértice no grafo, *n*, e ainda adiciona uma aresta a esses vértices, *m*.

O algoritmo de criação do percurso Euleriano, como o da criação do grafo, é também ele simples. É aqui onde usamos a *stack*. Este algoritmo recebe como parâmetros o grafo construído e um vértice de início. Este vértice pode ser o que quisermos. Os testes foram feitos com o vértice com a base AA, mas a implementação proposta irá constar que este parâmetro será o primeiro vértice do grafo. Voltando ao algoritmo, após criar a *stack* e a lista mutável onde vamos guardar o percurso, adicionamos o primeiro vértice à *stack* e entramos num ciclo *while*. Este ciclo *while* é executado enquanto a *stack* não esteja vazia. Dentro deste ciclo, vamos buscar o valor que está no topo da *stack* e de seguida vamos ver se o valor tirado é um vértice do grafo e, se for, verificamos se a lista das adjacências está vazia ou não. Se não estiver, vamos buscar o primeiro valor e vamos retirá-lo dessa lista e adicioná-lo à *stack*. Caso a lista esteja vazia, adicionamos este vértice à lista mutável que nos dá o caminho. Por fim, retornamos esta lista, mas só que por ordem contrária. Este algoritmo tem uma complexidade temporal *O(V+E)*, onde *E* representa o número total de arestas no grafo e *V* representa o número de vértices, uma vez que a função percorre cada aresta e cada vértice quando está a construir o caminho e tem uma complexidade espacial *O(V)*, em que *V* representa o número de vértices. Tem esta complexidade uma vez que a função armazena os vértices na *stack* e na lista mutável.

Dentro da função de execução destas funções, temos uma função que tem complexidade temporal *O(n)*, que é a função de leitura do ficheiro, em que *n* é o número de linhas do ficheiro. Depois, entramos dentro da função de escrita, em que chamamos a função que cria o grafo e, de seguida, temos um ciclo *for* que por cada elemento pertencente ao grafo, ele escreve o elemento e os vértices ao qual se liga. Depois de termos escrito as informações do grafo no ficheiro de *output*, passamos para o caminho. Como o elemento retornado da função é uma lista, fazemos simplesmente um *joinToString(separator:String)*. Esta função de extensão é o mesmo que fazer um ciclo *for* para cada elemento. Assim, ficamos com uma complexidade temporal total de *O(n*k+V+E)*, em que *n* vêm da função de criação do grafo e *V* e *E* veem da função de criação do percurso. A complexidade espacial é *O(n+m+V)*, em que *n+m* vem da função de criação do grafo e *V* vem da criação do percurso.

3. Avaliação Experimental

Para a avaliação experimental do problema, usámos o computador do Ricardo que é um portátil HP com um processador Intel i7 da 12ª geração e com 16 GB de RAM. Foram 10 testes para cada ficheiro, com diferentes quantidades de caracteres e com k igual a 3. Obtivemos os seguintes resultados:

3.1 *Ecoli – diferentes tamanhos do ficheiro*

Ecoli Completo	327423	302944	305001	331336	313373	320923	321706	324829	322392	322673
Ecoli Metade	28495	28025	28632	33511	28582	28999	28578	28635	28510	29208
Ecoli Um Quarto	6807	6809	7372	6991	6871	6751	6957	6772	6779	6671
Ecoli 2 Milhões	2560	1759	1727	1726	1775	1802	1745	1734	1755	1786
Ecoli 1 milhão	600	608	611	617	618	613	615	617	605	631

Tabela 1: Resultados do tempo de execução do programa, para os diferentes tamanhos. Com o ficheiro completo temos, em média, 319260 ms. Para metade dos caracteres, 29118 ms. Para um quarto dos caracteres, temos 6878 ms. Para 2 milhões caracteres 1837 ms e, finalmente, para 1 milhão de caracteres, 614 ms.

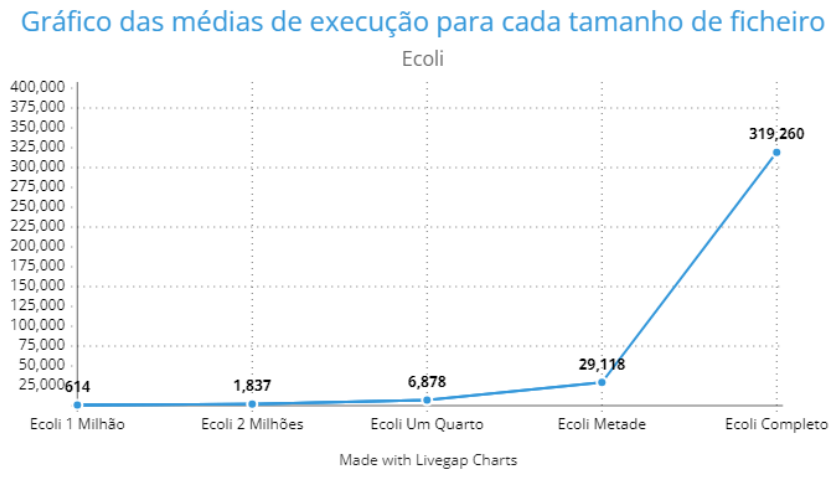

Gráfico 1: Correspondente ao valor médio de cada linha da tabela 1.

3.2 *Vibrio Cholerae – diferentes tamanhos do ficheiro*

Vibrio Cholerae Completo	6058	6080	6024	6014	6060	5949	6023	6078	5959	5923
Vibrio Cholerae Metade	1858	1807	1711	1801	1779	1743	1764	2053	1727	1782
Vibrio Cholerae Um Quarto	595	624	617	636	644	664	781	736	684	615
Vibrio Cholerae 500 mil	299	309	301	310	303	305	297	298	306	302
Vibrio Cholerae 250 mil	186	177	182	207	183	168	172	177	178	173

Tabela 2: Resultados do tempo de execução do programa, para os diferentes tamanhos. Com o ficheiro completo temos, em média, 6017 ms. Para metade dos caracteres, 1803 ms. Para um quarto dos caracteres, temos 660 ms. Para 500 mil caracteres 303 ms e, finalmente, para 250 mil caracteres, 180 ms.

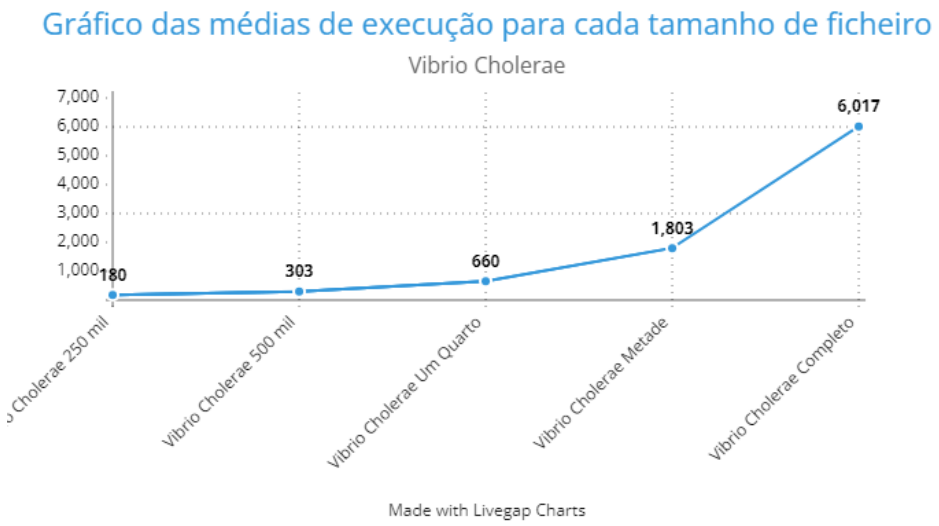

Gráfico 2: Correspondente ao valor médio de cada linha da tabela 2.

4. Conclusões

Concluindo, podemos ver através da análise dos tempos de execução que a nossa implementação é uma implementação que pode não estar muito bem otimizada. Isto porque para o ficheiro da bactéria *Ecoli* temos tempos de execução a chegarem aos 5 minutos.

Com isto pudemos aprofundar ainda mais a integração de estruturas de dados por nós implementadas, em vez de estruturas de dados dadas pelo *kotlin.collections*.

Referências

[1] "Disciplina: Algoritmos e Estruturas de Dados - 2324SV," Moodle 2023/24. [Online]. Available: https://2324.moodle.isel.pt. [Accessed: 07-04-2024].

[2] Introduction to Algorithms, 3º Edition. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. MIT Press.